Modeling and Simulation — Lesson 6
Synthesis: Choosing, Validating & Communicating Models

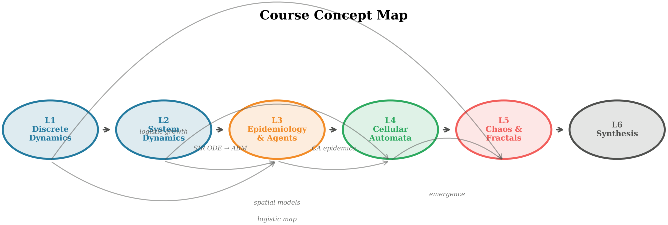
Václav B. Petrák

Faculty of Biomedical Engineering
Czech Technical University

March 19, 2026

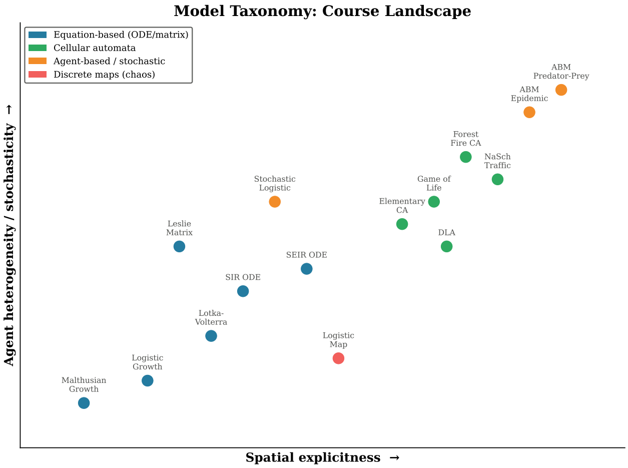
Model Taxonomy & Decision Framework

Looking Back: What Have We Built?



- Five lessons, each introducing a different **modelling paradigm**.
- Today we step back and ask: *when should I use which approach?*

The Modelling Landscape



- Two key axes: **spatial explicitness** and **agent heterogeneity**.
- Every model from the course occupies a different region.

Paradigm 1: Equation-Based Models

What: Ordinary differential equations (ODEs), difference equations, matrix models.

- Population is described by **aggregate variables** (S, I, R, N).
- Individuals are identical and well-mixed.
- Time can be continuous (ODE) or discrete (map / matrix).

Course examples:

- Malthusian & logistic growth (L1)
- Lotka-Volterra predator-prey (L2)
- SIR compartmental model (L3)
- Leslie matrix (L1)
- Logistic map (L5)

Paradigm 2: Cellular Automata

What: Grid of cells, each with a discrete state, updated by local rules.

- Space is **explicit**: a regular lattice.
- Time advances in discrete steps.
- **Emergence**: complex global patterns from simple local rules.
- No concept of individual "agents" — cells *are* the entities.

Course examples:

- Conway's Game of Life (L4)
- Forest fire model (L4)
- Elementary cellular automata (L4)
- Nagel-Schreckenberg traffic (project)

Paradigm 3: Agent-Based Models

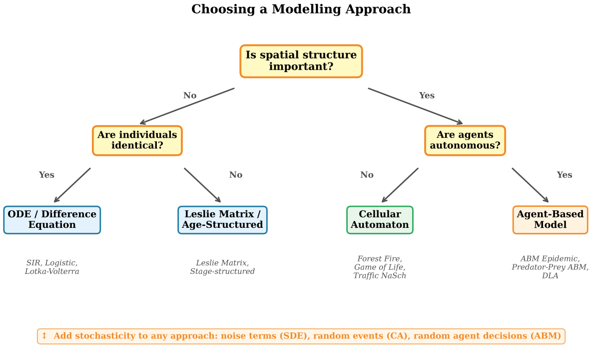
What: Individual agents with their own state, rules, and (often) position.

- Agents can be **heterogeneous**: different ages, speeds, infection status.
- Interactions depend on **proximity** or **contact networks**.
- Inherently **stochastic**: randomness in movement, infection, decisions.
- Most flexible — but hardest to analyse mathematically.

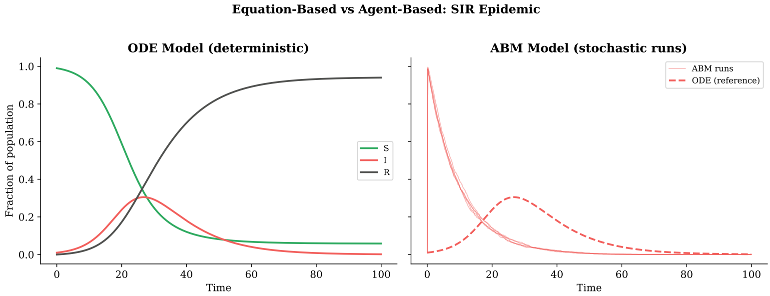
Course examples:

- Agent-based SIR epidemic (L3)
- Agent-based predator-prey (L2, project)
- Diffusion-limited aggregation (project)

Choosing a Modelling Approach



Comparing ODE vs ABM: Same System, Different Lens



- ODE: smooth, deterministic, fast to compute.
- ABM: noisy, captures individual variation, computationally heavier.
- **Key insight**: the ODE is the "mean-field limit" of the ABM as $N \to \infty$.

Trade-offs Summary

	ODE / Matrix	Cellular Automaton	ABM
Spatial structure	No	Grid	Flexible
Heterogeneity	No	Limited	High
Stochasticity	Optional (SDE)	Rule-based	Inherent
Analytical results	Often	Rarely	Rarely
Computational cost	Low	Medium	High
Interpretability	High	Medium	Lower

- There is no single "best" approach — it depends on your **research question**.
- A good strategy: start simple (ODE), add complexity only when needed.

Validation, Sensitivity & Common Pitfalls

Why Validation Matters

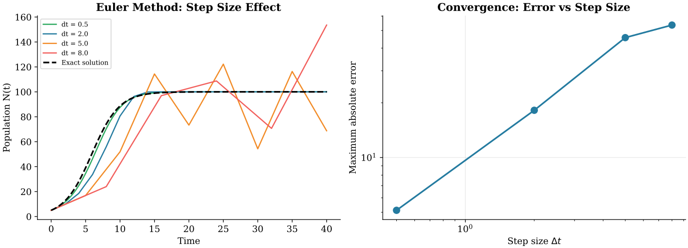
- A model is a **simplification** of reality — it is always “wrong” in some sense.
- The question is: *is the model useful for the intended purpose?*
- George Box (1976): **“All models are wrong, but some are useful.”**

Three levels of checking:

Verification	Does the code correctly implement the equations?
Validation	Does the model reproduce observed real-world behaviour?
Calibration	Can we find parameter values that fit data well?

Verification: Is the Code Correct?

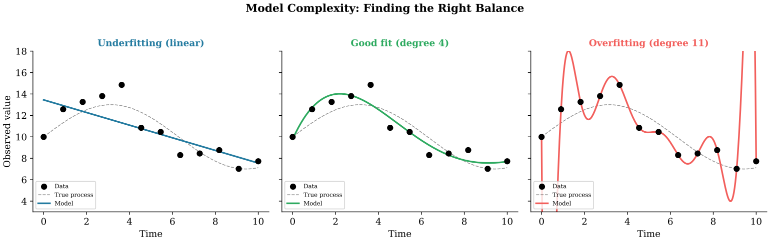
- **Conservation laws**: does $S + I + R = N$ hold at every step?
- **Limiting cases**: does the SIR reduce to exponential decay when $\beta = 0$?
- **Convergence**: does refining Δt improve the solution?
- **Comparison**: does the ABM match the ODE for large N?



Validation: Does the Model Match Reality?

- Compare model output against **independent data** not used for fitting.
- Qualitative: does the model produce the right *shape* of behaviour?
  - S-curves, oscillations, extinction thresholds, etc.
- Quantitative: compute error metrics (R^2, RMSE, AIC).
- **Important**: a model that fits training data perfectly may fail on new data.

The Overfitting Trap

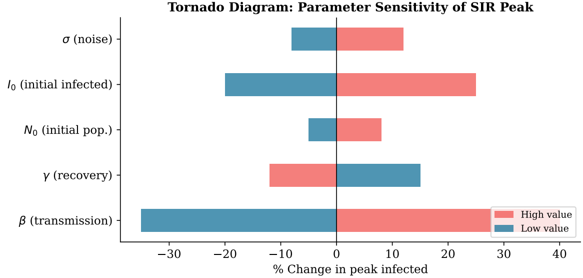


- **Underfitting**: model too simple, misses the pattern.
- **Overfitting**: model too complex, learns the noise.
- **Good fit**: captures the underlying process without chasing noise.
- Rule of thumb: *more parameters* $\neq$ *better model*.

Sensitivity Analysis

Question: How much does the output change when I vary a parameter?

- Vary one parameter at a time (OAT) while holding others fixed.
- Record a key output metric (e.g. peak infected, time to extinction).
- Visualise with a **tornado diagram**.



Sensitivity Analysis: Why It Matters

- Identifies **critical parameters** that drive the behaviour.
- Reveals which parameters need **careful measurement** vs. rough estimates.
- Helps communicate **uncertainty**: "If β increases by 10%, the peak doubles."
- For your projects:
  - Pick 2–3 key parameters.
  - Sweep each through a plausible range.
  - Show how a key output responds.

Common Modelling Pitfalls

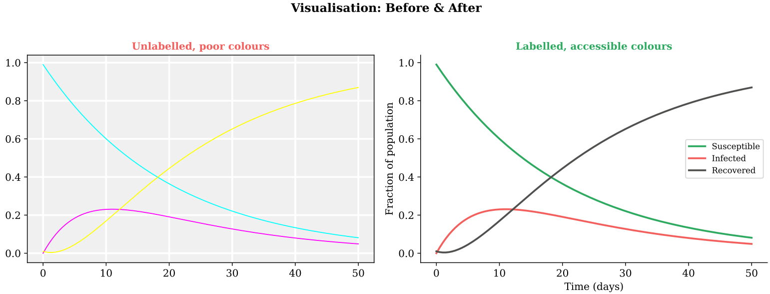
1. **Ignoring units.** Always track physical units (time^{-1}, individuals, etc.).
2. **Step size too large.** Euler's method can diverge or give wrong results.
3. **Too many free parameters.** If the model can fit anything, it explains nothing.
4. **No baseline comparison.** Always compare against a simpler model or null model.
5. **Cherry-picking runs.** In stochastic models, show *ensembles*, not single lucky trajectories.
6. **Confusing correlation with causation.** A good fit does not prove the mechanism is correct.

Stochastic Models: Ensemble Thinking

- A single ABM run tells you very little — it is **one sample** from many possible outcomes.
- Always run $N_{\text{runs}} \geq 20$ and report:
  - Mean trajectory
  - Confidence bands (e.g. 5th–95th percentile)
  - Distribution of key outcomes (peak size, final state)
- This is not a limitation — it is a **feature**: the spread tells you about risk and variability.

Communicating Results & Project Preparation

Visualisation Matters



- Your plots are the **first thing** reviewers and audiences see.
- Bad visualisation can hide correct results; good visualisation can illuminate them.

Visualisation Checklist

Every figure in your report should have:

1. **Axis labels** with units (e.g. "Time (days)", "Population *N*").
2. **Legend** or direct labels — the reader should never guess which line is which.
3. **Accessible colours** — avoid red/green only combinations (colour-blind readers).
4. **Caption** that describes what the reader should see.
5. **Consistent style** across all figures in the report.

Tools: `matplotlib` defaults are decent; use `plt.rcParams` to set a global style once.

Structuring Your Project Report

Recommended structure (4–6 pages + code):

1. **Introduction** What system are you modelling? Why is it interesting?
2. **Model Description** Equations, rules, parameters, assumptions. Include a diagram.
3. **Implementation** Key code design decisions. Not a code dump — highlight the important parts.
4. **Results** Figures with captions. What do the simulations show?
5. **Analysis** Sensitivity analysis, parameter sweeps, comparison to data or simpler models.
6. **Discussion** What did you learn? What are the limitations? What would you do next?

Writing About Models: Do's and Don'ts

Do:

- State all assumptions explicitly.
- Provide parameter values and their sources.
- Discuss what the model *cannot* capture.
- Compare against a baseline (simpler model, analytical solution, or data).

Don't:

- Claim the model "proves" something — models *suggest* or *are consistent with*.
- Present only the best-looking run out of many.
- Ignore parameters you did not vary.
- Skip units or dimensional analysis.

Presenting Simulations Live

For your final presentation (10–15 min):

1. **Hook** (1 min): What is the real-world problem?
2. **Model** (3 min): Show the key equations / rules. One diagram.
3. **Demo** (3 min): Run the simulation live or show an animation. This is the highlight.
4. **Results** (3 min): 2–3 key figures. What did you discover?
5. **Reflection** (2 min): What worked? What surprised you? What would you change?

Tip: prepare a **backup video/GIF** of the simulation in case the live demo fails.

Project Evaluation Criteria

Criterion	What we look for	Weight
Model correctness	Equations / rules are implemented correctly; code runs without errors	25%
Analysis depth	Parameter exploration, sensitivity analysis, comparison to baseline	25%
Visualisation	Clear, labelled, informative figures	20%
Report quality	Well-structured, assumptions stated, limitations discussed	15%
Presentation	Clear delivery, live demo, good slides	15%

Recap: The Modelling Workflow

1. **Define the question.** What behaviour do you want to understand or predict?
2. **Choose the paradigm.** ODE, CA, ABM — use the decision flowchart.
3. **Implement.** Start simple. Verify against known solutions.
4. **Explore.** Sweep parameters. Run ensembles. Compare to data.
5. **Validate.** Does it reproduce the qualitative / quantitative behaviour?
6. **Communicate.** Figures, report, presentation.

Good luck with your projects!